kiln: a container runtime for short-lived, untrusted AI agent workloads

White paper

Stephen Estrada

October 2026

Abstract

AI agents run code as a long series of short tool calls: a shell command, a test run, a script the model just wrote. Each call is untrusted, each one blocks the agent, and between calls the sandbox sits idle. kiln is a Linux container runtime built for that pattern. In randomized measurements across one ARM Lima VM, an idle-gap sweep showed why an absolute "cold-start" number is misleading: after 5 s idle, kiln and bare crun converged at 14.9 and 14.8 ms, while Docker took 140.9 ms. In a separate campaign across five fresh boots, at a defined 100 ms gap, Docker's lifecycle was 25.8× kiln's (95% session-bootstrap interval 23.8–27.8×). A fit over 0–100 idle sandboxes estimated kiln at 748.8 KiB per sandbox against 5,007.2 KiB for Docker. Across three boots, Docker's ratio narrowed from 21.4× for a tiny shell workload to 2.2× for an HTTP GET, as fixed overhead was amortized. Non-networked starts need no shared daemon. kiln provides a user namespace, seccomp, a per-sandbox disk quota and optional egress policy by default. These are product-level comparisons of the tested stacks, not component benchmarks; `true` measures runtime overhead, not end-to-end agent speed.

Table 1: Primary v2 results on one 4-vCPU ARM VM. Lifecycle values are p50 from the 5 s point of one-boot idle-gap sweeps (ms); throughput is the mean of three 64-client batches (successful completions/s); memory is the median slope of three fits over 0–100 sandboxes (KiB/sandbox). These rows are exploratory until replicated on another host. Details in §4.

	kiln	Docker	Podman	nerdctl
5 s idle, no network (ms)	14.9	140.9	109.3	147.5
5 s network, all-runtime sweep (ms)	25.5	191.3	—	230.7
5 s network, paired rerun (ms)	23.4	—	205.9	—
64 clients, no network (/s)	836.6	6.9	15	10.5
Memory slope (KiB)	748.8	5,007.2	1,203.2	13,171.8
Shared daemon, no network	none	two	none	containerd

1 The problem: agents pay the overhead on every step

An agent works in a loop: the model decides on an action, a tool runs it, the result goes back to the model, and the model decides again. Tools that touch the system (run a command, install a package, execute generated code) need a sandbox, because the code was written by a model or downloaded from somewhere nobody checked. Three properties of this workload decide what a sandbox runtime should optimize:

1. **Latency multiplies by the number of steps.** The loop blocks on every tool call, so whatever the runtime adds is paid per step, not per session. For illustration only: over 50 tool calls, the measured runtime overhead of `docker exec` accumulates to 50 × 53.7 ms ≈ 2.7 s, and that of `kiln exec` to 50 × 3.5 ms ≈ 0.18 s. That is accumulated overhead, not the end-to-end time of the task, which also includes the model and the commands.
2. **Sandboxes are mostly idle.** Between calls the model is thinking, often for seconds. What a sandbox costs while idle, times the number of concurrent sessions, is what a host pays.
3. **The code is untrusted.** Isolation has to be the default, not a flag someone remembers, and it should not have to be traded for latency.

How fast the code runs once started should not depend on the runtime: a container is a set of kernel features around ordinary processes, so a workload runs on the same kernel whichever runtime set it up. The CPU microbenchmark observed no difference; the I/O one was inconclusive (§4.7). A runtime's lever is the cost around the code: starting, idling, running the next command and tearing down.

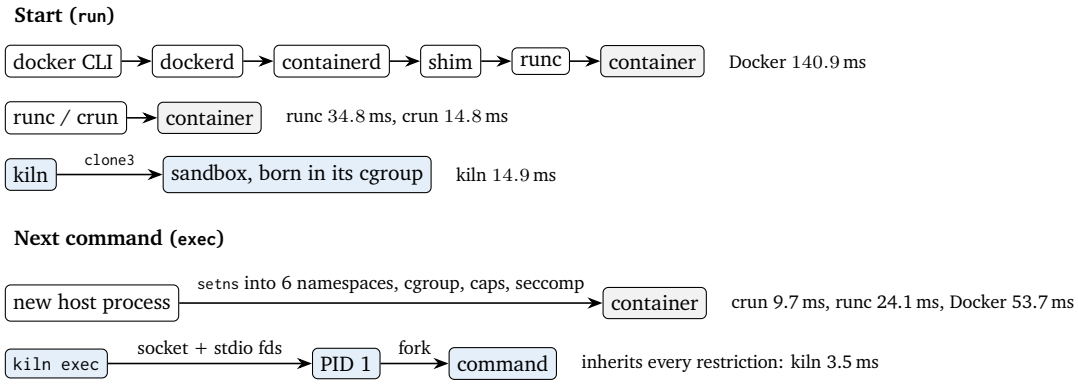


Figure 1: The path from a request to running code. Docker crosses an API and two daemons; runc and crun are single processes. For the next command in an existing sandbox, the others start a host process that joins the container from outside; kiln asks the sandbox's own PID 1 to fork it. Times are p50 of the one-shot 5 s-idle v2 sweep and the legacy exec series in §4; those protocols are not directly comparable.

2 Why general-purpose runtimes pay for things agents don't need

General-purpose runtimes are built for long-running services managed by people and orchestrators, and their costs follow from that (Figure 1).

Daemon layers. `docker run` goes from the CLI to `dockerd` over its API, to `containerd`, to a per-container shim, and finally to runc, which does the actual work. Those layers buy remote management, logging drivers, restart policies and a stable API, none of which a sandbox for one tool call uses. They show up in the 5 s-idle one-shot lifecycle (140.9 ms for Docker against 34.8 ms for runc, which Docker uses underneath, though Docker also prepares an overlay snapshot and a seccomp profile, so the gap is not the daemons' alone), as resident memory (`dockerd` and `containerd` together use about 160.1 MiB PSS whether one container runs or none), and as about 11 MiB of private memory per container for its Go shim and PID 1 together (11228 KiB USS).

Generic image handling. Docker and containerd manage OCI images: content-addressed layers, snapshotters, registries. That machinery exists to distribute images between machines. An agent sandbox starts from one image that is already on the host, so kiln uses that image directly as the read-only lower layer of an overlay, with a per-sandbox writable layer on top.

Joining from outside. To run another command in an existing container, `runc exec` and `crun exec` start a new host process that enters the container's namespaces one by one with `setns`, joins its cgroup, sets capabilities and seccomp, and only then executes the command. runc, a Go program, joins the cgroup by writing to `cgroup.procs`, which migrates a running process and waits for an RCU grace period; crun creates the process inside the cgroup with `clone3`. Docker adds its daemon path on top of runc.

Grace periods on the critical path. Making the kernel's RCU grace periods expedited, a host-wide setting, cut runc's cold start from 33.6 ms to 27.3 ms and kiln's former build's from 29.0 ms to 17.0 ms: both were waiting for grace periods. Current kiln and crun are unaffected (13.2 ms vs 13.3 ms; crun 13.2 ms vs 13.4 ms).

3 Design choices that buy speed

No mandatory shared daemon. `kiln run` is one process that sets up the sandbox, waits for it and removes it. All state lives in the kernel and a few directories, named by a run ID. A non-networked start needs no shared daemon; networked sandboxes use kiln-dns, a small fixed service, for DNS. A persistent sandbox (`kiln create`) gets a small supervisor of its own, a shim, which owns its cleanup; the shim and the sandbox's PID 1 run the same binary and share almost all their pages, so the two together cost 136 KiB of private memory (USS) against Docker's 11228 KiB for its shim and PID 1.

Born in the cgroup. kiln creates the sandbox with `clone3(CLONE_INTO_CGROUP)`, so the child exists inside its cgroup from its first instruction and no running process is ever migrated. The older approach, writing the PID to `cgroup.procs`, cost 5–15 ms of grace-period waiting on every cold start and was the largest single step in kiln's own before-and-after (Figure 3).

Table 2: What selected cases set up in the benchmarks. Podman and nerdctl are described with the full matrix in `docs/benchmarks.md`.

	kiln	`docker run --rm`	runc / crun (bare spec)
Path to the kernel	one process	CLI, dockerd, containerd, shim, runc	one process
Root filesystem	overlay on XFS loop device, per-run quota	overlay snapshot on ext4	unpacked tree on ext4, read-only
Namespaces	PID, mount, UTS, IPC, net, cgroup, **user**	same, no user ns	same, no user ns
cgroup limits	memory, swap, CPU, PIDs, OOM group kill	memory, no swap, CPU, PIDs	memory, swap, CPU, PIDs
Syscall filter	seccomp denylist, `no_new_privs`	default seccomp profile	none
PID 1	kiln-init (signals, reaping)	the command	the command
Network (networked cases)	veth, bridge, nft chain, DNS policy	veth on docker0, iptables NAT	loopback only

Talking to the kernel directly. Starting a helper program costs milliseconds before it does anything: a cold `nft` takes 3.9 ms just to load its libraries. kiln configures links, addresses and routes with its own rtnetlink client instead of `ip` and `nsenter`, runs exactly one `nft` per networked start (its policy batch ends with a listing that is checked against the shared base ruleset, and repairs are committed only on drift), sends teardown deletions as one nfnetlink batch whose socket is closed by a short-lived child so kiln does not wait for the kernel's deferred freeing, and leaves the host end of the veth pair to the dying network namespace.

Exec through PID 1. A persistent sandbox's PID 1 holds a Unix socket, opened before the sandbox's root was swapped. `kiln exec` sends the arguments and passes its stdin, stdout and stderr as file descriptors; PID 1 forks the command onto them and returns its exit code. The command is born with every namespace, the cgroup, the seccomp filter and the Landlock policy, because it inherits them. Nothing joins from outside, so there is nothing to re-apply.

Security on by default, inside the measured time. Every number in this paper includes what kiln sets up by default and the others, as benchmarked, did not (Table 2): a user namespace that makes sandbox root an unprivileged host UID, a seccomp filter with `no_new_privs`, a per-sandbox XFS disk quota, and an init process that forwards signals and reaps orphans. Networked sandboxes get an nftables egress policy (no private ranges with `--net full`; only listed destinations with `--allow`) and a DNS forwarder that resolves only allowed names. A Landlock read-only mode is available as an option; it was not part of the benchmark runs.

4 Results

Method. The v2 harness times direct argument vectors from immediately before `Popen` until `waitpid`: setup, the command and the runtime's synchronous cleanup through CLI exit. Kernel-deferred destruction, such as a veth tied to a dying namespace, can finish after the timer. Output goes to files, not pipes. Each measured round runs every case once; seeded randomized Latin squares balance each case's position. Ten warm-up rounds precede 100 measured rounds. Five fresh VM boots are the replication units, and the 95% intervals below bootstrap the geometric mean of per-boot median ratios. Individual-invocation p95 and p99 values are not confidence intervals.

Every runtime starts the same imported Alpine tree under 512 MiB memory with no swap, 1 CPU and 128 PIDs. The image is already local. `true` is an overhead microbenchmark: it does no useful work and therefore exposes the fixed lifecycle cost. It is not time-to-ready, an image pull, cold caches or evidence that application code runs faster. The end-user stack tier is kiln, Docker 29.1.3, Podman 4.9.3 and nerdctl 2.4.1/containerd 2.2.1; runc 1.3.4, crun 1.14.1 and youki 0.7.0 are a low-level floor handed a prepared OCI bundle; gVisor runsc is reported separately because it interposes a userspace kernel.

The host was an M2 MacBook Air under normal use. The guest was a Lima `vz` VM with 4 vCPUs and 4 GiB, Ubuntu 24.04 and Linux 6.8 aarch64. Raw JSONL, copied configs, executable and rootfs hashes, versions, kernel boot IDs and environment manifests accompany every run.

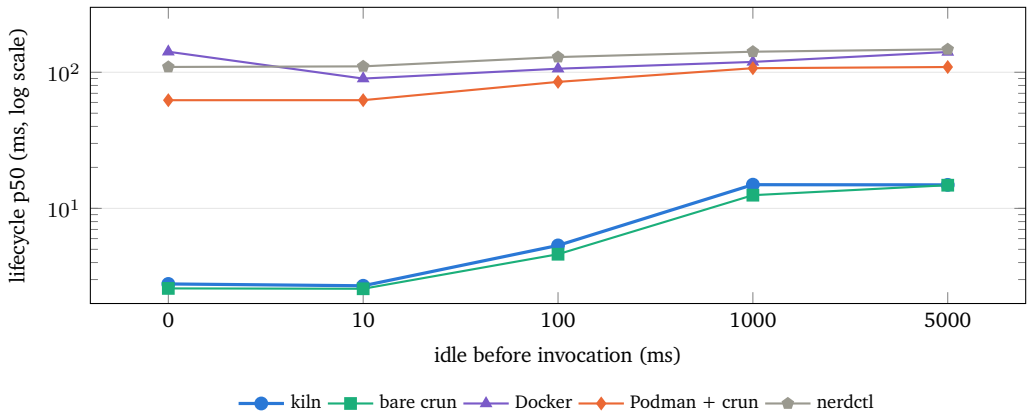


Figure 2: One-shot `true` across idle gaps, 10 measured runs per point in one boot. The 100 ms point is not a stable "cold" state: kiln and crun still benefit from recently active kernel and VM paths. This sweep is exploratory, not a confidence interval.

4.1 Idle time changes the answer

At zero idle, kiln and crun take 2.8 and 2.6 ms. At 5 s they converge again at 14.9 and 14.8 ms, while Docker, Podman with crun and nerdctl take 140.9, 109.3 and 147.5 ms. With their normal network paths at 5 s, kiln takes 25.5 ms and Docker 191.3 ms. In the separate corrected Podman pair, kiln took 23.4 ms and Podman 205.9 ms. These are ten-run points from one boot per sweep, so they establish the idle-gap effect and approximate scale, not a universal cold-start constant.

4.2 Replicated ratios at a defined 100 ms gap

Across five fresh boots, Docker's non-networked lifecycle was 25.8× kiln's (95% CI 23.8–27.8×), Podman with crun was 17.5× and nerdctl was 27.6×. On the networked paths Docker was 16.8× (15.4–18.3×) and nerdctl 21× kiln. The corrected Podman network comparison was 20.4× (20.1–20.8×).

The low-level floor explains the architecture: crun's ratio was 0.95× (95% CI 0.87–0.99×), a small absolute difference that changes with idle state. kiln tracks crun; it does not consistently beat it. runc and youki were 4.2× and 3.6× kiln. runsc was 8.4×, but belongs to the stronger-isolation tier rather than the shared-kernel product comparison.

Where kiln's time went. Before this work, kiln took 28.9 ms cold without a network and 87.9 ms with one; back to back, the same build took 2.7 ms, which is why measuring cold mattered. Figure 3 shows each change, all builds measured in one session. Two causes explain almost all of the old time: waiting for RCU grace periods (migrating into the cgroup, closing nftables sockets after deleting commits, deleting the veth) and starting helper processes. What remains without a network is mostly creating the namespaces (1.6 ms), the sandbox's mounts (2.5 ms), starting the binary, and about 5 ms between the command exiting and the host seeing the sandbox gone; a network adds one `nft` process and the veth (Table 3).

Table 3: Where a cold start's time goes: `KILN_TRACE` phases, medians of 15 cold runs in ms, before and after this work (from `docs/benchmarks.md`).

	`--net none`		`--net full`	
Phase	before	now	before	now
Image check, storage, quota, overlay directories	1.2	1.5	1.5	1.3
cgroup create	0.6	0.9	0.8	0.9
Clone with five namespaces	1.3	1.6	1.6	1.6
Move the child into its cgroup	**8.6**	gone	**9.9**	gone
Network setup on the host	4.5	—	**42.3**	7.5
Child setup (mounts, `pivot_root`, `/proc`, user ns)	~2	~2.5	~2	~2.4
Command runs and exits	1.4	1.6	0.9	1.0
Sandbox exit seen by the host	5.0	5.3	5.1	5.3
Network cleanup	0.1	0.1	**21.4**	0.3
cgroup and run directory cleanup	0.3	0.3	0.2	0.3

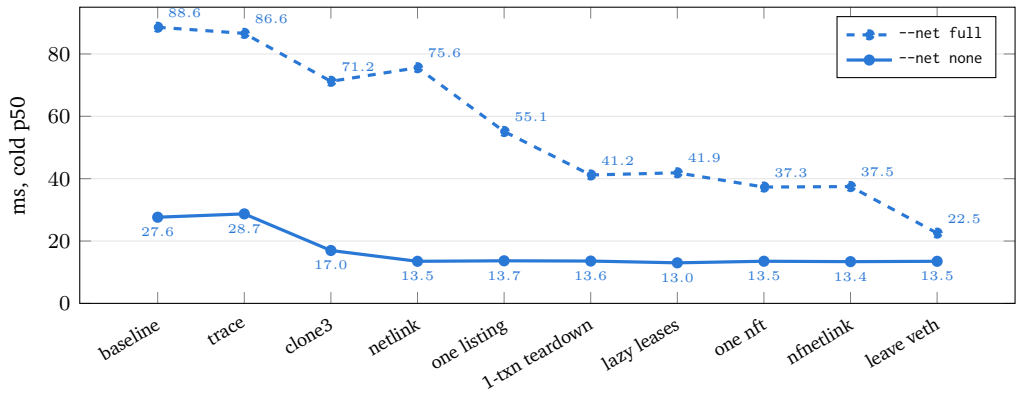


Figure 3: Cold-start p50 after each change, every build measured in the same session (`bench/results/steps`). Some changes only show in combination; `docs/benchmarks.md` explains each.

4.3 Warm start: the next command (legacy series)

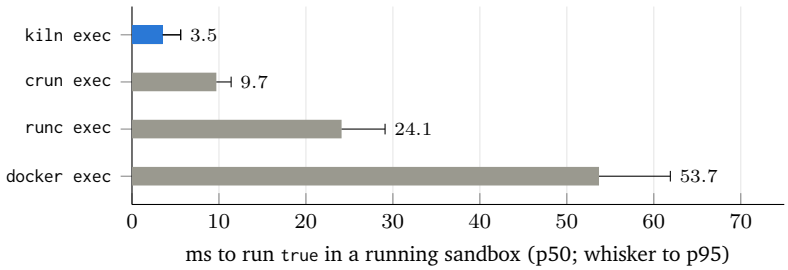


Figure 4: Warm start, 100 runs each with 100 ms idle before each run. This is the earlier one-session fixed-order series, retained because v2 did not repeat the exec path. kiln: a `kiln create` sandbox; others: a container running `sleep infinity`.

For an agent session the sandbox is created once and each tool call runs in it, so warm start is the runtime overhead an agent pays on almost every step. `kiln exec` takes 3.5 ms (p99 5.9 ms), against 9.7 ms for crun (p99 12.0 ms), 24.1 ms for runc (p99 33.6 ms) and 53.7 ms for Docker (p99 63.3 ms), Figure 4: 2.8×, 6.9× and 15.3× kiln's time. With a network the numbers barely move (kiln 3.8 ms, Docker 54.1 ms). The difference is the path in Figure 1: a fork inside the sandbox against a new process joining from outside.

4.4 Concurrency: kiln stays near the crun floor

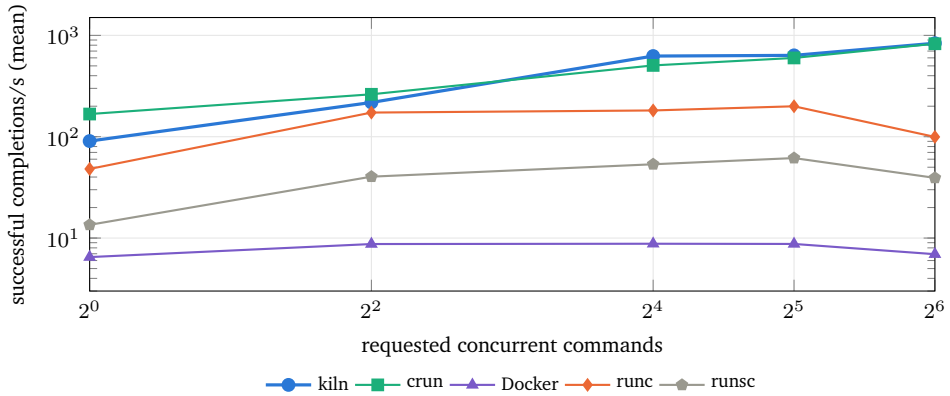


Figure 5: One exploratory boot, three batches per level. Throughput is successful completions divided by batch wall time. Python threads release a barrier together, but starts are not perfectly simultaneous: at level 64, kiln without networking spread 15.7–20.4 ms; all original cases ranged from 12.3 ms to 7.87 s, with the widest spread in one Docker network batch.

At the requested 64-client non-networked level, kiln averaged 836.6 completions/s and crun 827; Docker, Podman with crun and nerdctl averaged 6.9, 15 and 10.5. kiln completed every invocation, with p50 44.6 ms and p95 205.8 ms. The three kiln batches varied widely (296.9–1,243.2 completions/s), so this establishes a promising saturation profile, not a stable capacity number. youki timed out 22 invocations across levels 4–64 and is excluded from throughput comparisons. Every other runtime completed all commands.

A separate corrected network pair averaged 114.4 completions/s for kiln and 10.6 for Podman at 64 clients, again in three batches on one boot. Its first attempt exposed a real kiln race: one of 351 invocations failed when an older network namespace removed a reused veth between an `EEXIST` response and stale-link deletion. That entire attempt is preserved but excluded. Commit `2cca44a` treats only the resulting `ENODEV` as already-cleaned state and adds a 160-start concurrent stress check; the fresh-boot replacement completed all 702 invocations across the pair without a failure or timeout.

4.5 Representative one-shot workloads

The same four end-user stacks ran six small tool-like commands from one identical expanded rootfs, with three fresh-boot sessions per workload. The non-networked cases used a 100 ms gap and 40 measured rounds per session; DNS and HTTP used a 1 s gap and 15 rounds because they contact public infrastructure. Table 4 reports the geometric mean, over boots, of each stack's median divided by kiln's. The three-session bootstrap intervals are coarse, and DNS/HTTP include resolver and Internet variance.

Table 4: End-to-end lifecycle ratio to kiln for representative workloads (three fresh-boot sessions per workload; lower is better, kiln = 1.0).

Workload	Docker	Podman + crun	nerdctl
Shell + SHA-256	21.36	15.86	24.16
Python + JSON/SHA-256	4.67	3.66	5.38
Node + JSON/SHA-256	3.81	3.02	4.28
Write + hash 4 MiB	3.29	2.64	3.82
DNS lookup	4.16	5.01	5.4
HTTP GET	2.15	2.5	2.6

The expected amortization is visible: Docker's ratio falls from 21.36× for the tiny shell command to 4.67× for Python, 3.81× for Node and 2.15× for HTTP. kiln is reducing the fixed cost around the command, not accelerating the work itself. The filesystem case still uses different backing storage. DNS was the noisiest: Docker's coarse interval was 3.21–6.86×, so its point estimate should not be read as a precise ranking.

4.6 Idle memory slope

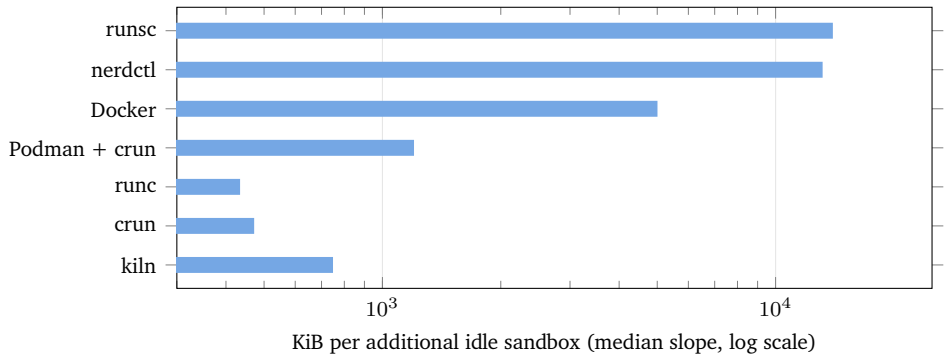


Figure 6: OLS slope of host memory in use against 0, 10, 25, 50 and 100 idle sandboxes. Three randomized fits in one boot. Pre-existing daemon memory cancels in each before/after delta and is outside the fit; the intercept is count-independent change and noise. The chart shows median slopes.

kiln's median slope was 748.8 KiB per sandbox (range 680–836.6), against 5,007.2 KiB for Docker (6.7× kiln), 1,203.2 KiB for Podman and 13,171.8 KiB for nerdctl. The v2 host-wide fit includes kernel objects and supersedes the earlier two-batch estimate of 370–379 KiB. It is still one boot with three fits, not a confidence interval. Bare crun and runc used 471.8 and 434.5 KiB; runsc used 13,986.4 KiB. Docker's shared daemons and nerdctl's containerd are fixed costs outside these slopes. kiln has no mandatory shared daemon for non-networked starts; kiln-dns is a small fixed service for networked sandboxes, and each persistent sandbox keeps its own shim and PID 1.

In the separate corrected network pair, kiln's median slope was 1,101.2 KiB (range 550.2–1,444.1) and Podman's was 1,470.7 KiB (914.4–1,843.8). The three-fit ranges overlap, so this run does not establish a network-memory ranking.

4.7 Steady state: no CPU difference observed

Two single-threaded workloads under the same 1-CPU limit, timed inside the sandbox so start-up is excluded, 7 runs each, interleaved: hashing 512 MiB of zeros, and writing 1 GiB to the sandbox's writable layer with `fsync`. No CPU difference was observed in the 7 runs. The I/O result is inconclusive and not configuration-equivalent (Table 5): the ranges overlap widely, because the VM's disk is a file on the Mac's SSD, and the writes land on different storage. kiln's upper layer is an overlay on XFS in a loop device, for its disk quota; Docker's is an ext4-backed overlay snapshot; runc and crun write directly to the unpacked tree on ext4. This test cannot separate the runtime from the storage under it.

Table 5: Steady state: median seconds (min–max) over 7 runs.

	kiln	crun	runc	Docker
CPU-bound	2.93 (2.92–2.94)	2.93 (2.90–3.01)	2.93 (2.92–2.94)	2.94 (2.92–3.00)
I/O-bound	1.87 (1.60–2.20)	2.01 (1.74–2.90)	1.66 (1.38–1.77)	1.71 (1.34–2.04)

5 Fairness and limitations

- **One machine, one VM.** Every number comes from a Lima VM (4 vCPU, 4 GiB) on an M2 MacBook Air: aarch64, Linux 6.8, Ubuntu 24.04. The Mac was otherwise in normal use. The primary 100 ms latency ratios use five fresh boots; representative workloads use three. The idle-gap sweep, concurrency and memory slope each use one boot and are exploratory. Nothing was measured on x86-64 or bare metal, and kiln's author ran the benchmarks.
- **End-user stacks, not components.** Table 2 lists what each runtime sets up as configured. kiln, Docker, Podman and nerdctl are complete user-facing stacks, but their image, overlay, networking, seccomp and daemon paths still differ. runc, crun and youki receive a prepared bundle and are only a low-level floor. runsc offers stronger isolation. These comparisons cannot assign a difference to one component.

- **Matched inputs, not identical work.** v2 imports one rootfs tar into each stack and gives every case 512 MiB memory with no swap, 1 CPU and 128 PIDs. kiln additionally creates a user namespace and per-sandbox XFS quota. Storage paths differ: kiln uses overlayfs on an XFS loop device; image stacks use their configured snapshotters; bare runtimes use the unpacked ext4 tree. The legacy warm and optimization series predate v2 and did not give Docker an explicit no-swap flag.
- **No universal "cold" number.** Figure 2 shows that latency changes materially between 0, 100 and 1000 ms idle. The paper therefore names the gap with every lifecycle result. The five-boot confidence intervals apply to ratios at 100 ms, not to the one-boot 5 s values.
- **Sampling limits.** p95 and p99 describe individual invocations. Only the five-boot latency campaign has a five-session interval; the workload intervals have three sessions and are coarse. Three concurrency batches and three memory fits in one boot expose behavior but do not establish host-to-host capacity or confidence intervals.
- **An overhead microbenchmark.** `true` isolates the runtime's cost. The representative shell, Python, Node, filesystem, DNS and HTTP cases add small real workloads, but they remain synthetic one-shot commands. DNS and HTTP also include public resolver and Internet variance.
- **Runtime coverage.** The suite measured Docker, Podman with crun and runc, nerdctl/containerd, bare runc, crun and youki, and gVisor runsc. The VM does not expose `/dev/kvm`, so Firecracker and Kata were not runnable; they remain a separate stronger-isolation tier for a KVM host.
- **Scope.** kiln has no OCI image or runtime-spec compatibility, runs on a single host, and has no orchestrator integration yet. Its isolation is a shared-kernel container's: a kernel bug reachable through an allowed syscall is still an escape.

6 Future work

- **Pre-warmed pools.** A daemon that creates sandboxes ahead of time and hands one out on request would turn a cold start into an exec: 3.5 ms in the legacy exec series instead of a full lifecycle. Networked pool members could have their veth pair and firewall chain prepared ahead too. This needs the planned daemon and a policy for pool size.
- **Smaller steps already measured.** Loading libnftables in-process instead of starting `nft` measured 0.55 ms to load and 1.2 ms to apply a policy, against 4–6 ms for the process: 3–4 ms per networked start. Returning the exit code before teardown would save most of the 5 ms exit phase, at the cost of cleanup finishing after `kiln run` returns.
- **Snapshot and restore** of a prepared sandbox (installed packages, warm files) so a session can start from where a previous one left off; the overlay's upper layer already records exactly what a session changed.
- **Multiple hosts**: a daemon per node and a Kubernetes operator that places sandboxes, with the same per-node design.
- **Next measurements.** Repeat v2 on the intended Mac Mini and on an x86-64 bare-metal host; replicate concurrency and memory across boots; match the storage and overlay path where possible; add package installation, larger images and long-lived exec-heavy agent traces; and run Firecracker and Kata on a host that exposes KVM. An independent operator should reproduce at least the headline campaign. The committed traces reproduce the current analysis, but a byte-for-byte measurement rerun also needs the rootfs tarballs identified by hash in the manifests; publish them as release artifacts first.

7 Conclusion

A sandbox for agents is judged on the cost around the code: how fast it starts, how it behaves under many simultaneous calls, and what it costs while it waits. On one ARM VM, with identical rootfs contents and randomized case order, kiln tracked bare crun's lifecycle floor while remaining far below the tested Docker, Podman and nerdctl stacks in lifecycle overhead and marginal idle memory. It did so while turning on a user namespace, seccomp and a disk quota by default. The result is architectural evidence, not a universal speed claim: idle state changes the absolute number, most experiments still need another host, and kiln shares the host kernel. The raw records and manifests make those boundaries auditable; exact measurement replication additionally requires the separately archived rootfs artifacts named by their hashes.